Task 1: Phase-Based Feature Extraction

A short, simple guide for the “Who Said Stop?” GMM-UBM project

1. What is the task?

Turn each audio recording into a sequence of numbers that describe useful properties of the speaker’s voice. These numbers are called **features**. The important rule in this competition is that the model-input features must come from the **phase** of the Fourier transform, not from its magnitude.

In plain words: audio is the raw material; feature extraction converts it into numbers that the later GMM system can learn from.

2. The basic processing steps

1. Load the audio	Read the recording and make sure it uses the required sample rate and format.
2. Split it into frames	Cut the audio into short, usually overlapping pieces. A suggested example is a 25 ms window and a 10 ms hop.
3. Apply a window	Apply a window function (for example, Hamming) to each frame to reduce edge effects.
4. Apply the Fourier transform	Use an STFT/FFT on each frame. The result is complex-valued and contains both magnitude and phase.
5. Build phase-based features	Use a chosen phase method, such as direct phase, group delay, phase difference, or temporal phase difference.
6. Remove silent frames	Ignore frames that contain little or no speech. Energy/magnitude may be used for this purpose only, not as model-input features.
7. Normalise and return features	Apply a suitable, consistent normalisation. You may also add temporal derivatives if useful.

3. Magnitude vs phase: the key idea

The Fourier transform gives a complex value for each frequency. It can be described using two parts:

Magnitude	How strong a frequency component is.
Phase	The angle (position in its cycle) of that frequency component.

Your project requires phase-derived features as the model input. Do **not** use magnitude spectrograms, log-mel spectrograms, or magnitude-based MFCCs as input features. Magnitude or energy is allowed only for silence detection or numerical stabilisation, and you must document that use.

4. Which phase feature can you choose?

Direct phase	Use phase values from the Fourier transform.
Group delay	Describe how phase changes as frequency changes.
Phase difference	Compare phase values between frequency components.
Temporal phase	Describe how phase changes from one frame to the next.
Other options	Phase spectrograms, group-delay cepstra, baseband phase difference, relative phase shift, LP-residual phase features, or learned phase encodings.

You do not need to implement every option. Choose a suitable method, explain how it works, and clearly describe how you handle phase wrapping (angles jumping between $-\pi$ and $+\pi$), low-energy frequency bins, and silence.

You may apply a Mel or Bark filterbank to a suitable phase-derived representation, or combine multiple feature streams. Be careful: phase angles wrap around, so do not blindly average raw phase angles.

5. What does “same pipeline for every split” mean?

Use the same feature definition and processing settings for **background-training**, **enrolment**, **validation/development**, and **trial/evaluation** recordings. For example, do not use one frame size for enrolment and a different one for trials. Otherwise, the feature values may not be comparable.

If you learn a transformation or calculate dataset-level normalisation statistics, fit them using only permitted background-training data. Do not fit them using held-out trial recordings.

6. Coding rules and what to deliver

Implement feature extraction using **PyTorch (torch) and torchaudio only**. Do not use NumPy, SciPy, librosa, scikit-learn, or other libraries for core feature extraction. You can use Python’s standard library for file handling and configuration.

Keep the code modular: write small reusable functions or classes for steps such as framing, phase calculation, silence removal, and normalisation. The same feature extractor should be reusable for every dataset split.

By the end of Task 1, document:

- Which phase-based feature method you chose and how it is calculated.
- Frame length, hop length, sample rate, FFT size, and other settings.
- How you handle phase wrapping, low-energy bins, and silent recordings.
- How you normalise features and the final feature dimension.
- A modular, working PyTorch/torchaudio feature extractor.

Keep this in mind: Task 1 does not identify the speaker yet. It only turns each recording into consistent, phase-based feature vectors. The GMM-UBM system uses those vectors in later tasks.